

第24回 実践：Web ゲームプログラミング 3（弾）

- 前回：キーボードで自機を動かした
- 今回：スペースキーで弾を撃つ
- 次回：敵を出し、画面外のを消す

実習時のフォルダ構成

Web_名前 の中に 24 フォルダを作る。第 23 回の sample フォルダと exercise フォルダを、24 の中にコピーして、続きから作る。

```
Web_名前
├── 24
│   ├── sample          ← 第 23 回の sample をコピーする
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 23 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、sample/main.js に書き足す）
- exercise：自分のシューティングゲーム（演習 1～2、発展）
- index.html と style.css は、変更しない。

講義内容

sample/main.js に書き足す。書いたら sample/index.html をブラウザで開いて確認する。

1. 前回の復習

- `addEventListener("keydown", function (event) { })` で、キーが押されたときの処理を登録する。
- キーが押されているかどうかを `isLeftPressed` などの変数に記録し、実際の移動は `updatePlayer` で行う。

今回は、スペースキーで弾を撃てるようにする。弾は何発も同時に画面に出るので、第 19 回で学んだ配列を使う。

配列と `for` の復習：

- 配列[番号] で要素を取り出す。番号は 0 から。要素の数は `配列.length`。
- `for (let i = 0; i < 配列.length; i++) { }` で、すべての要素を順に処理できる。

2. 今回の main.js の並び

```
// ===== 準備 =====  
// ===== ゲームの状態 =====  
// ===== 自機・弾 ===== ← 見出しを変え、弾の変数を追加する  
// ===== キー入力 ===== ← スペースキーを追加する  
// ===== 更新 ===== ← updateBullets を追加し、update から呼び出す  
// ===== 描画 ===== ← drawBullets を追加し、draw から呼び出す  
// ===== ゲームループ =====
```

3. 弾をオブジェクトの配列で表す

1 発の弾は、自機と同じように { x, y, w, h, speed } のオブジェクトで表す。何発もの弾を、1 つの配列に入れて管理する。

```
bullets = [ { x: 198, y: 440, w: 4, h: 12, speed: 8 }, ← bullets[0] (1 発目)  
            { x: 198, y: 360, w: 4, h: 12, speed: 8 } ] ← bullets[1] (2 発目)
```

- `bullets[0]` は 1 発目の弾 (オブジェクト)。`bullets[0].y` で、1 発目の弾の y 座標が取り出せる。
- `for` 文の `i` を使うと、`bullets[i].y` のように、すべての弾を順に処理できる。

「自機」の部分の見出しを `// ===== 自機・弾 =====` に変えて、弾の変数を 2 つ追加する。

```
// ===== 自機・弾 =====  
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };  
let bullets = [];  
let shotTimer = 0; // 次の弾を撃てるまでのフレーム数
```

- `[]` は、要素が 1 つもない、空の配列。弾を撃つたびに、ここに弾を追加していく。
- `bullets` は、あとの回で「空の配列を代入し直す」ことがあるので、`let` で作る (第 22 回のルール)。

4. スペースキーを記録する

「キー入力」の部分を、次のように書き換える (第 23 回で書いた `console.log(event.key);` の行は、ここで消す)。

```
// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;
let isSpacePressed = false;

document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
  if (event.key === " ") {
    isSpacePressed = false;
  }
});
```

- スペースキーの `event.key` は " " (空白 1 文字)。

5. 弾を追加する・動かす (updateBullets)

「更新」の部分の `updatePlayer` の下に、`updateBullets` を追加する。

```
function updateBullets() {
  if (shotTimer > 0) {
    shotTimer -= 1;
  }
  // スペースキーを押していて、撃てる状態なら、弾を追加する
  if (isSpacePressed && shotTimer === 0) {
    bullets.push({ x: player.x + player.w / 2 - 2, y: player.y, w: 4, h: 12, speed: 8 });
    shotTimer = 10;
  }
  // すべての弾を上へ動かす
  for (let i = 0; i < bullets.length; i++) {
    bullets[i].y -= bullets[i].speed;
  }
}
```

`updateBullets` は、次の 3 つのことを順に行っている。

(1) 連射の間隔 (shotTimer)

- `shotTimer` は、次の弾を撃てるまでの待ち時間 (フレーム数)。

- 弾を撃つと 10 になり、1 フレームごとに 1 ずつ減る。0 になると、また撃てる。
- これがないと、スペースキーを押している間、毎フレーム（1 秒に 60 発）弾が出てしまう。

(2) push で弾を追加する

- 配列 `push(値)`：配列の最後に、要素を 1 つ追加する。例：`[10, 20]` に `push(30)` をすると `[10, 20, 30]` になる。
- ここでは、新しい弾のオブジェクトを追加している。弾の `x` は、自機の中央 (`player.x + player.w / 2`) から、弾の幅の半分 (2) を引いた位置にする。

(3) すべての弾を動かす

- 第 19 回と同じ `for` 文で、すべての弾の `y` 座標を `speed` だけ減らす（上へ動かす）。第 19 回で数値の配列 `bulletYs[i] -= 50` として練習したことを、オブジェクトの配列で行っている。

「更新」の部分の `update` を、次のようにする。

```
function update() {  
  frameCount += 1;  
  updatePlayer();  
  updateBullets();  
}
```

6. 弾を描く (drawBullets)

「描画」の部分の `drawPlayer` の下に `drawBullets` を追加し、`draw` から呼び出す。

```
function drawBullets() {  
  ctx.fillStyle = "#ffff00";  
  for (let i = 0; i < bullets.length; i++) {  
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);  
  }  
}  
  
function draw() {  
  drawBackground();  
  drawPlayer();  
  drawBullets();  
}
```

7. 画面の外に出た弾はどうなっているか

スペースキーで何発か撃ってから、コンソールに `bullets.length` と入力して `Enter` キーを押す。何回か入力すると、画面の上に消えた弾の数も含めて、どんどん増えていく。

- 画面の外に出た弾も、配列の中には残り続けている。
- このままでは、見えない弾が配列にたまり、処理がだんだん重くなる。
- 画面の外に出た弾を配列から消す方法は、次回に学ぶ。

8. うまく動かないとき

症状	確認すること
弾が出ない	<code>isSpacePressed</code> を <code>keydown</code> で <code>true</code> にしているか。 <code>update</code> から <code>updateBullets</code> を、 <code>draw</code> から <code>drawBullets</code> を呼んでいるか
弾が止まっている	<code>for</code> の中で <code>bullets[i].y -= bullets[i].speed;</code> をしているか
弾がつながって出る	<code>shotTimer = 10;</code> を書いたか
弾が自機の中央から出ない	<code>x</code> の計算 (<code>player.x + player.w / 2 - 弾の幅の半分</code>)

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 弾を撃てるようにする

- 講義と同じように、スペースキーで弾を撃てるようにする。弾の色・大きさ・速さは、自分で決める。
- 弾が自機の中央から出るように、`x` 座標を調整する（弾の幅を変えたときは、引く値も変える）。

演習 2 連射の間隔を確かめる

- スペースキーを押し続けたとき、`shotTimer = 30;` にすると 1 秒に約何発出るかを予測してから（1 秒は約 60 フレーム）、実際に確かめる。
- `shotTimer` の値を 3 などにも変えて、連射の間隔がどう変わるかを確かめる。最後に、自分がよいと思う値にする。
- コンソールに `bullets.length` と入力して、弾の数が増え続けていることを確かめる。

発展（任意）

- 1 回撃つと、自機の中央に加えて、左端と右端からも弾が出るようにする。

この回の終わりの `sample/main.js` 全体

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機・弾 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
let bullets = [];
let shotTimer = 0;           // 次の弾を撃てるまでのフレーム数

// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;
let isSpacePressed = false;

document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
  if (event.key === " ") {
    isSpacePressed = false;
  }
});

// ===== 更新 =====
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
```

```

    player.x = canvas.width - player.w;
  }
}

function updateBullets() {
  if (shotTimer > 0) {
    shotTimer -= 1;
  }
  // スペースキーを押していて、撃てる状態なら、弾を追加する
  if (isSpacePressed && shotTimer === 0) {
    bullets.push({ x: player.x + player.w / 2 - 2, y: player.y, w: 4, h: 12, speed: 8 });
    shotTimer = 10;
  }
  // すべての弾を上へ動かす
  for (let i = 0; i < bullets.length; i++) {
    bullets[i].y -= bullets[i].speed;
  }
}

function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
}

// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function drawBullets() {
  ctx.fillStyle = "#ffff00";
  for (let i = 0; i < bullets.length; i++) {
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
  }
}

function draw() {
  drawBackground();
  drawPlayer();
  drawBullets();
}

// ===== ゲームループ =====
function gameLoop() {
  update();
  draw();
  requestAnimationFrame(gameLoop);
}

gameLoop();

```